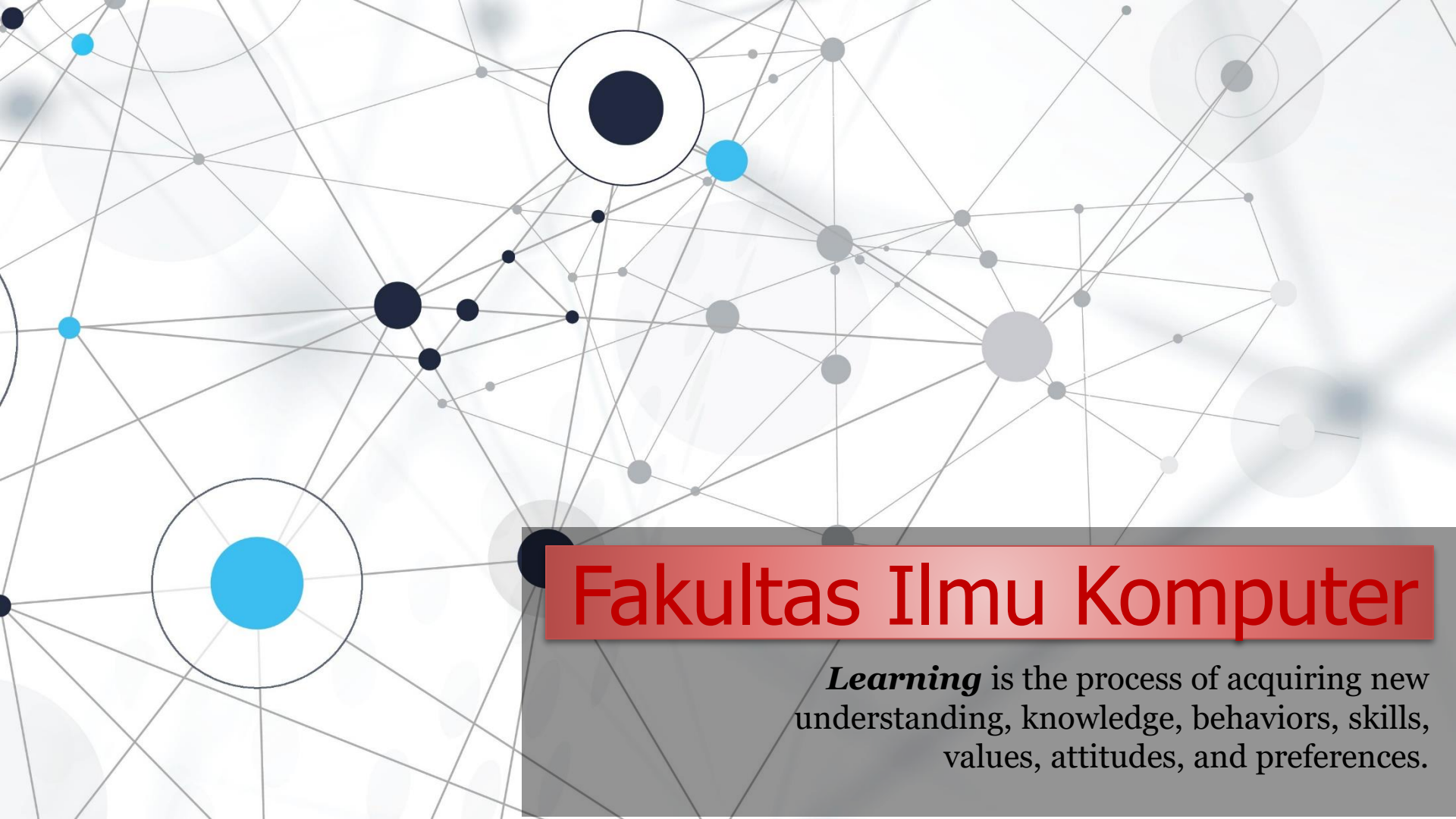


An abstract network diagram with various sized nodes (black, blue, grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

Fakultas Ilmu Komputer

Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

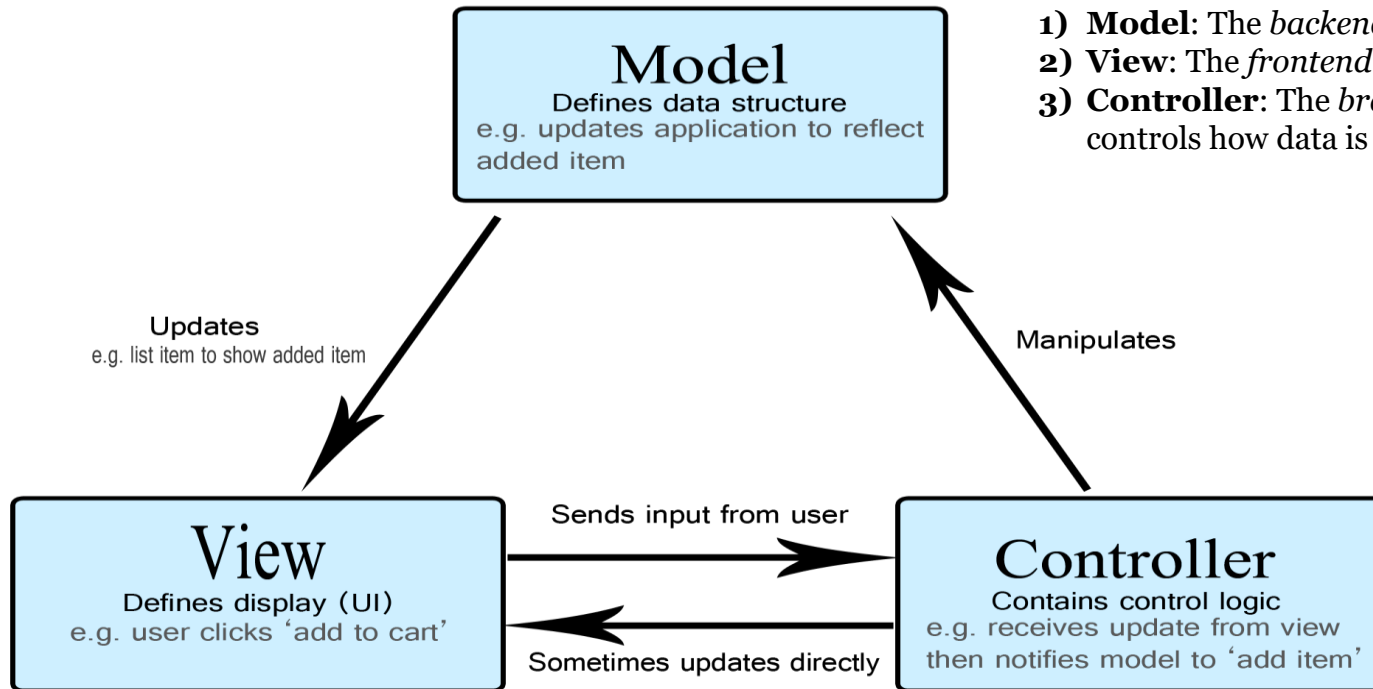
Back-end Framework #3

Default Controller & Default Method



Defining a Default Controller and Method

What is a Controller?



- 1) **Model**: The *backend* that contains all the data logic.
- 2) **View**: The *frontend* or graphical user interface (GUI).
- 3) **Controller**: The *brains* of the application that controls how data is displayed.

The Model

The model defines what data the app should contain. If the state of this data changes, then the model will usually notify the view (so the display can change as needed) and sometimes the controller (if different logic is needed to control the updated view).

The View

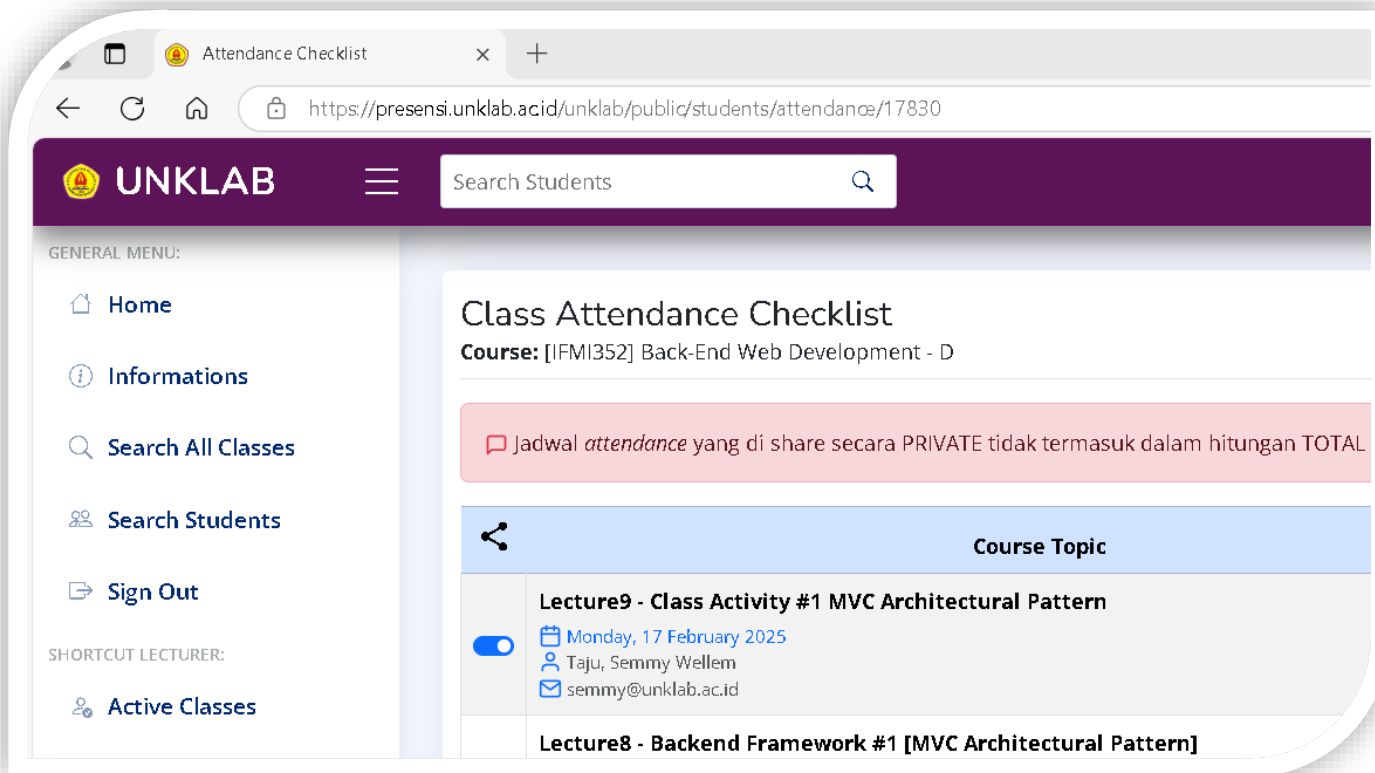
The view defines how the app's data should be displayed.

The Controller

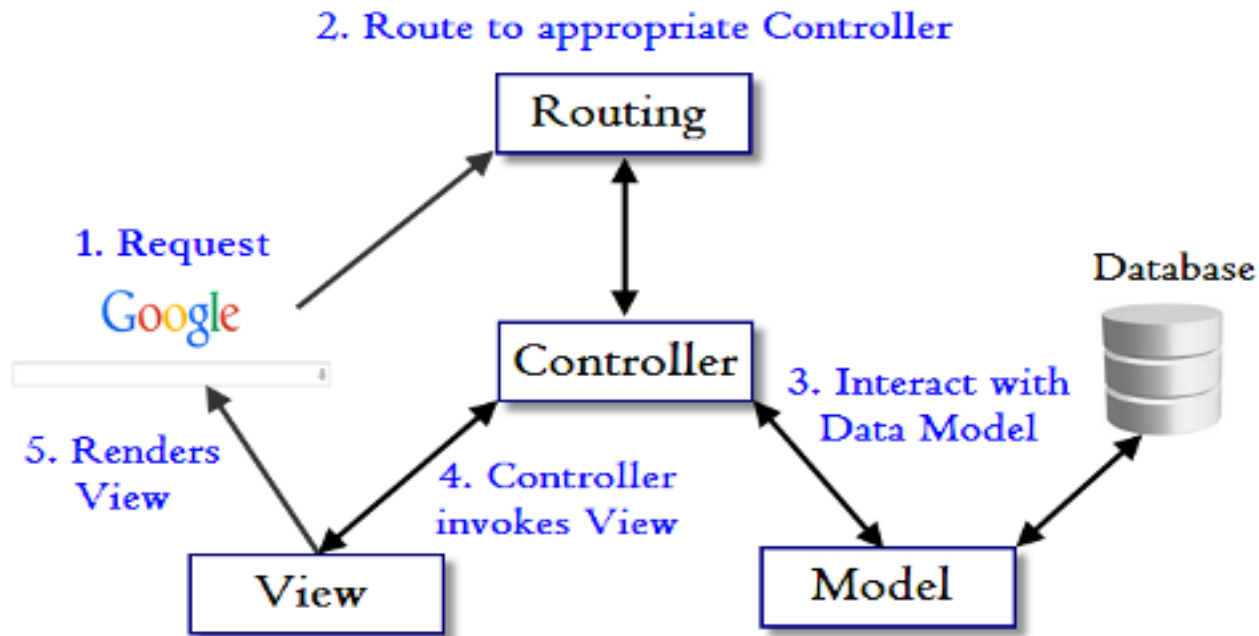
The controller contains logic that updates the model and/or view in response to input from the users of the app.

What is a Controller?

- 1) **Controllers** are the **HEART/BRAIN** of our application, as they **determine how HTTP requests should be handled**.
- 2) A **Controller** is simply **a class file** (in OOP) that is named in a way that can be **associated with a URL**.

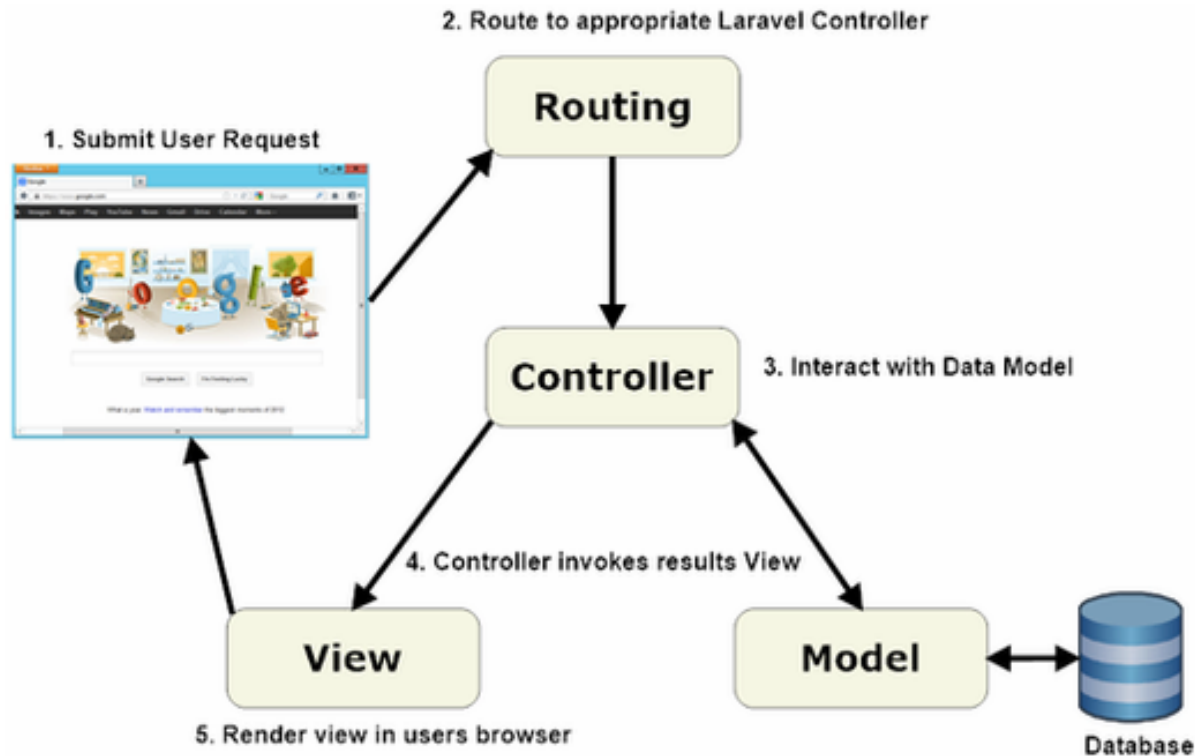


What is a Controller?



Normally a **user interacts with the view** and performs some action on the screen. A *request* is generated from the *view* which is handled by the **controller**. **Controller** is actually responsible for handling *http request*.

Laravel Controller



MVC URL Routing system allows to create powerful and flexible URLs for the applications.

It provides two functionalities:

- 1) Inspect incoming URLs** - This functionality manifests when the application receives a client request.
- 2) Generate outgoing URLs** - This functionality manifests when we render URLs using `Url Action` in the View.



1 – Some Useful Functions

[We'll use those functions later]

1 - PHP file_exists() Function

The **file_exists()** function checks whether a file or directory exists. **Note:** The result of this function is cached. Use clearstatcache() to clear the cache.

```
<?php
```

```
// check whether file exists or not
```

```
$file_pointer = '/user01/work/Home.php';
```

```
if (file_exists($file_pointer)) {  
    echo "The file $file_pointer exists";
```

```
}else{  
    echo "The file $file_pointer does not exists";  
}
```

```
?>
```


2 - PHP unset() Function

The **unset()** function destroys a given variable. Here we use unset() function to delete a given element index in array.

```
$a = array(1,2,3,4,5);  
unset($a[2]);  
print_r($a);
```

```
Array  
(  
    [0] => 1  
    [1] => 2  
    [3] => 4  
    [4] => 5  
)
```

Index 2 already removed.

3 - PHP method_exists() Function

It checks if the class method exists in the given object. It returns TRUE if the method given by method_name has been defined for the given object, FALSE otherwise.

```
<!DOCTYPE html>
<html>
<body>
    <?php
        class Fruit {
            // Properties
            public $name;
            public $color;

            // Method set name
            function set_name($name) {
                $this->name = $name;
            }

            // Method get name
            function get_name() {
                return $this->name;
            }
        }

        // Create object
        $obj = new Fruit();

        if(method_exists($obj, 'get_name')){
            echo 'Method exist.';
        }else{
            echo 'Method not exist.';
        }
    ?>
</body>
</html>
```

Method exist.

Sr.No	Parameter & Description
1	object(Required) The tested object
2	method_name(Required) The method name.

4 - PHP array_values() Function

The array_values() function returns an array containing all the values of an array.

Tip: The returned array will have numeric keys, starting at 0 and increase by 1.

```
// PHP function to illustrate the use of array_values()
function Return_Values($array)
{
    return (array_values($array));
}

$array = array("ram"=>25, "krishna"=>10, "aakash"=>20, "gaurav");
print_r(Return_Values($array));
?>
```

Output:

```
Array
(
    [0] => 25
    [1] => 10
    [2] => 20
    [3] => gaurav
)
```

5 - call_user_func_array() Function

The **call_user_func_array()** function calls a user function given with an array of parameters. The call_user_func_array() function can call a custom function *"function"* with the parameters from "param_arr array".

Syntax

```
mixed call_user_func_array( callback function [, array param_arr])
```

```
<?php
    function Box($width,$height, $depth) {
        $b = $width*$height*$depth;
        echo $b;
    }
    call_user_func_array("Box", array("width" => 10, "height" => 20, "depth" => 30));
?>
```

Output

```
6000
```

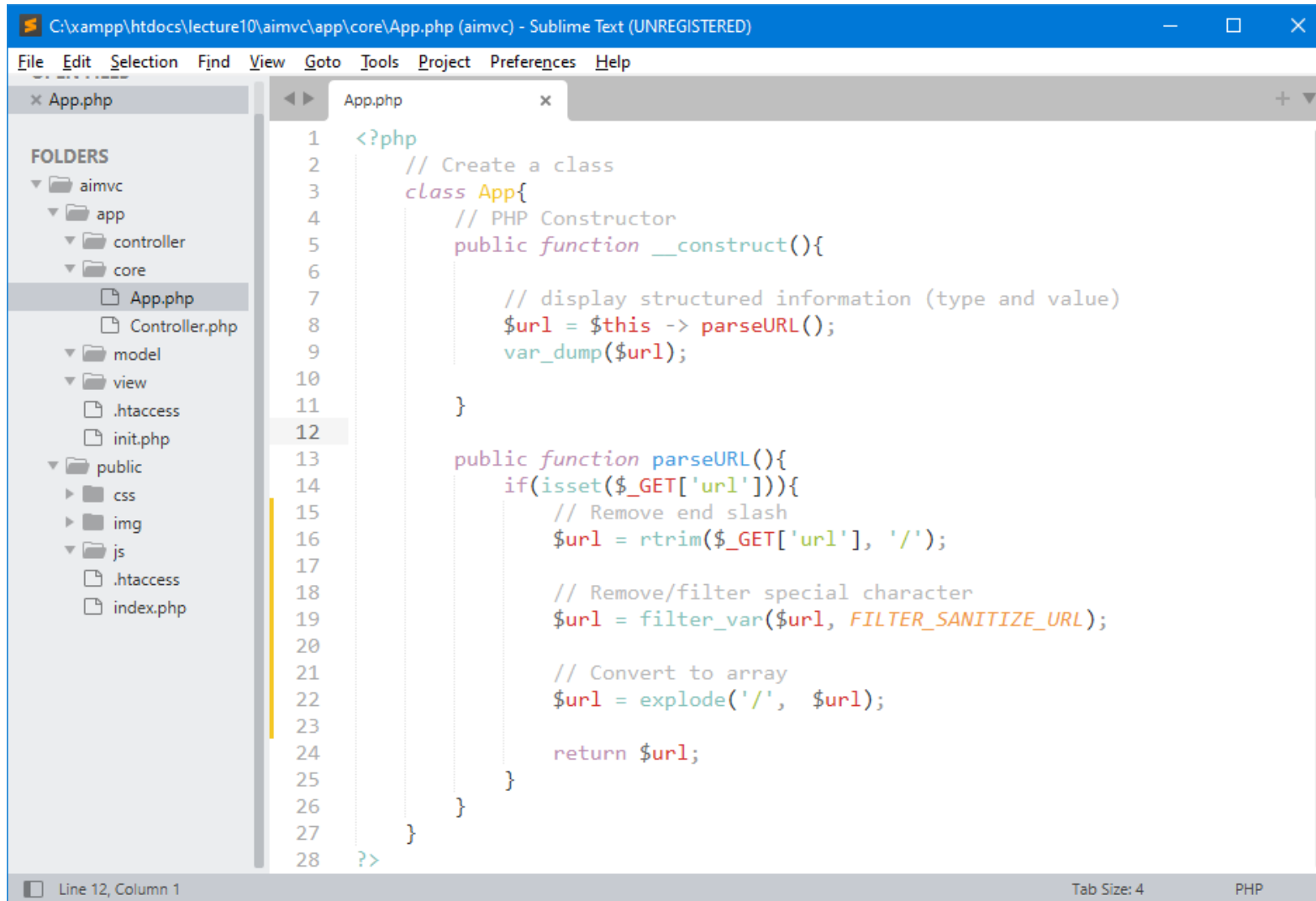
A decorative border of green leaves and branches at the top of the slide.

2 – Check Routing Rules *from Previous Lecture*

A decorative border of green leaves and branches at the bottom of the slide.

Handling Request URL

Handle more *routing rules* when *users* try to request through URL.



```
1 <?php
2 // Create a class
3 class App{
4 // PHP Constructor
5 public function __construct(){
6
7 // display structured information (type and value)
8 $url = $this -> parseURL();
9 var_dump($url);
10
11 }
12
13 public function parseURL(){
14 if(isset($_GET['url'])){
15 // Remove end slash
16 $url = rtrim($_GET['url'], '/');
17
18 // Remove/filter special character
19 $url = filter_var($url, FILTER_SANITIZE_URL);
20
21 // Convert to array
22 $url = explode('/', $url);
23
24 return $url;
25 }
26 }
27 }
28 ?>
```

URL Routing

Typically, there is a one-to-one relationship (*hubungan satu-ke-satu*) between a URL string and its corresponding controller class/method. The segments in a URL (*bagian dalam URL*) normally follow this pattern:

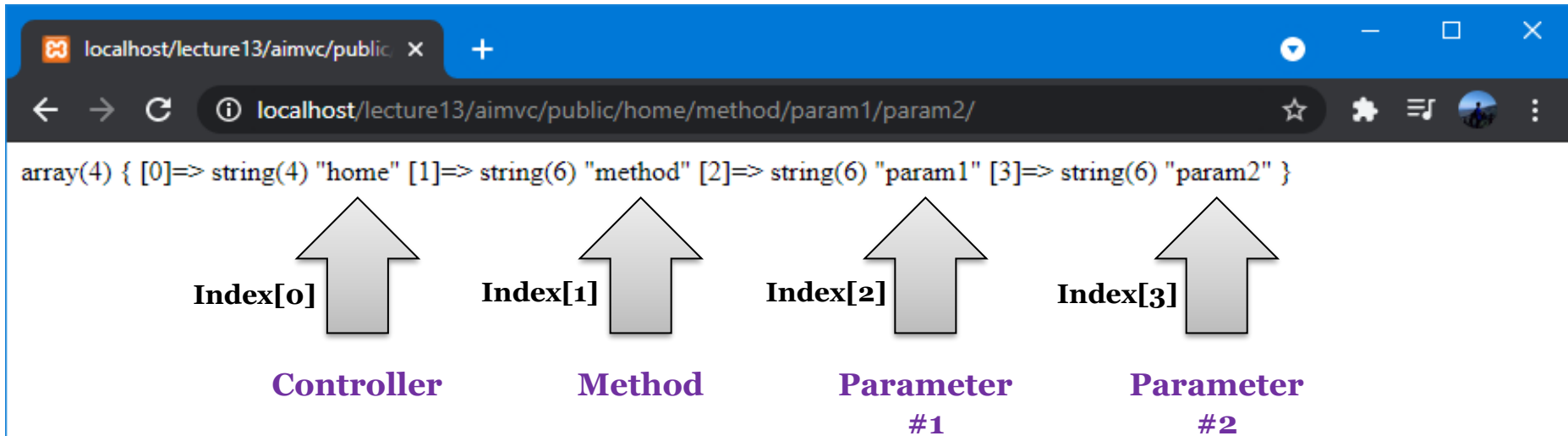
`http://www.example.com/class/method/parameter1/parameter2/`

What is a Controller?

A Controller is simply a class file that is named in a way that can be associated with a URI.

Class merupakan *controller class* yang sudah dibuat dan biasanya segmen kedua (**method**) dari URL disiapkan untuk nama *metode*.

URI Routing *from* Previous Demo

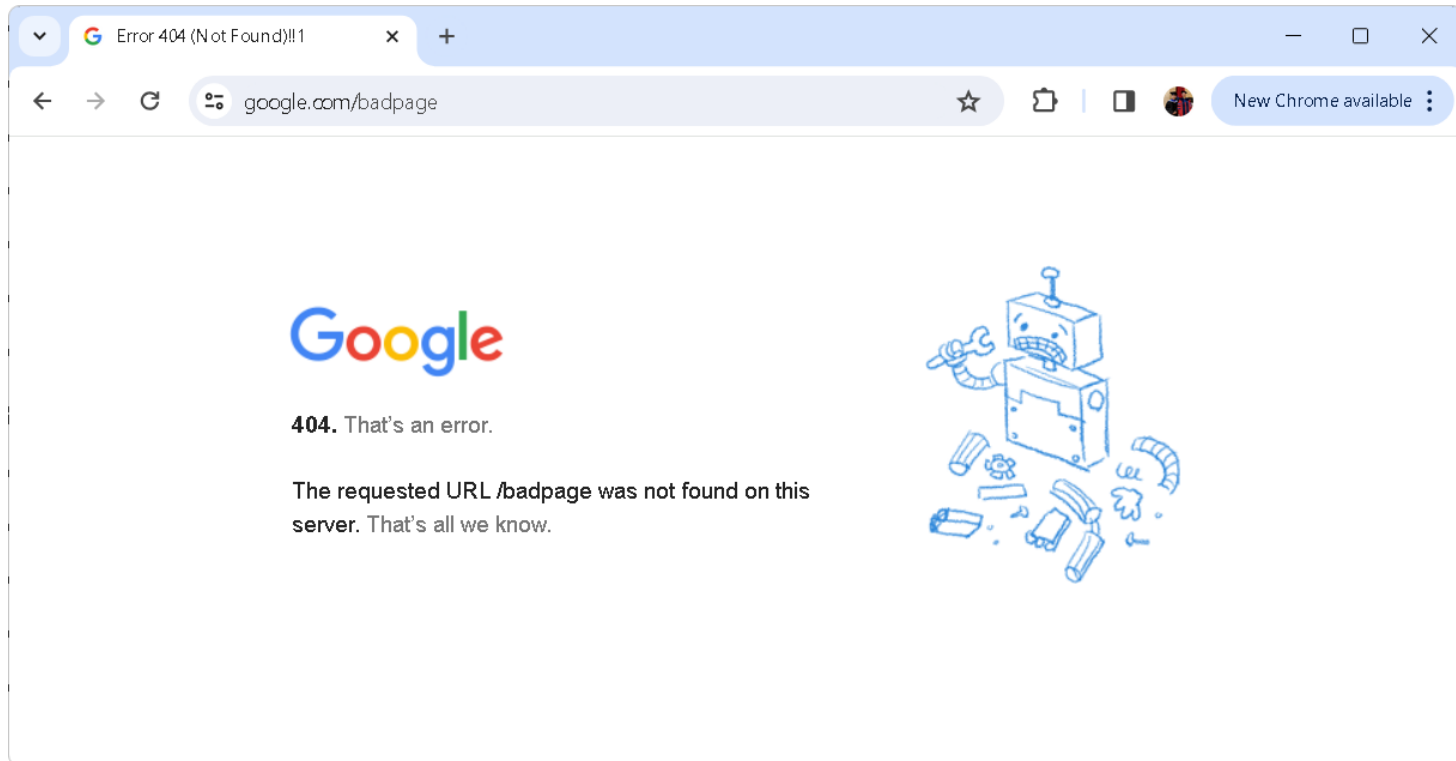


Our app starts from this directory location:
<http://localhost/lecture13/aimvc/public/>

This means Controller default (home) and method default (index).

URL Format: <http://localhost/lecture13/aimvc/public/class/method/param1/param2/>

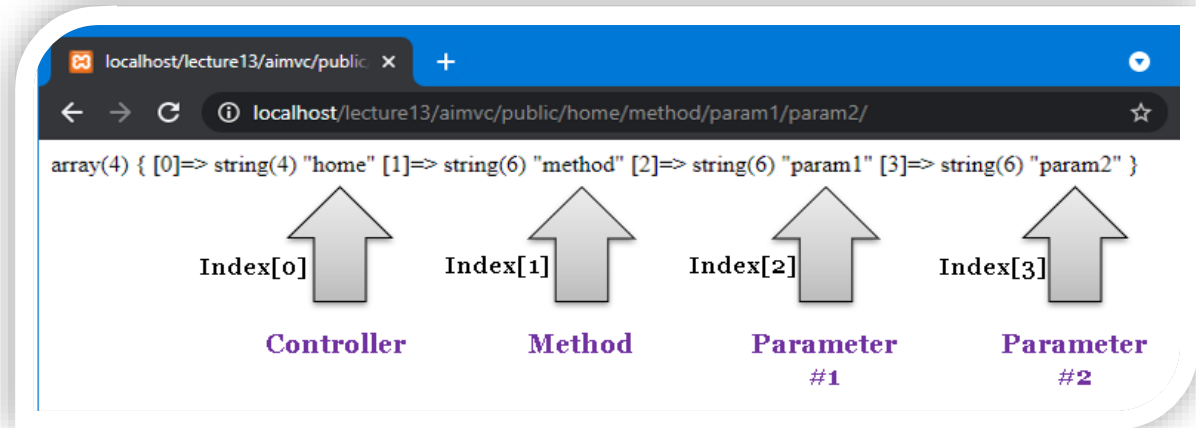
Bagaimana cara untuk handle "Error 404 Not Found" pada request URL yang tidak ditemukan di server?



Default Controller vs Default Method

In MVC (Model-View-Controller) design using PHP, the term "Default Controller" or "Default Method" is **a way to handle HTTP requests and determine how they are processed by default** if no controller or method is specified directly.

Keep in mind that,
The use of default controllers and default methods helps in organizing the application logic and ensuring that the user gets the right response.



Default Controller adalah controller yang dipanggil oleh framework jika tidak ada controller yang ditentukan dalam URL request.

Default Method adalah method/fungsi dalam controller default yang dipanggil jika tidak ada metode spesifik yang ditentukan dalam URL request.



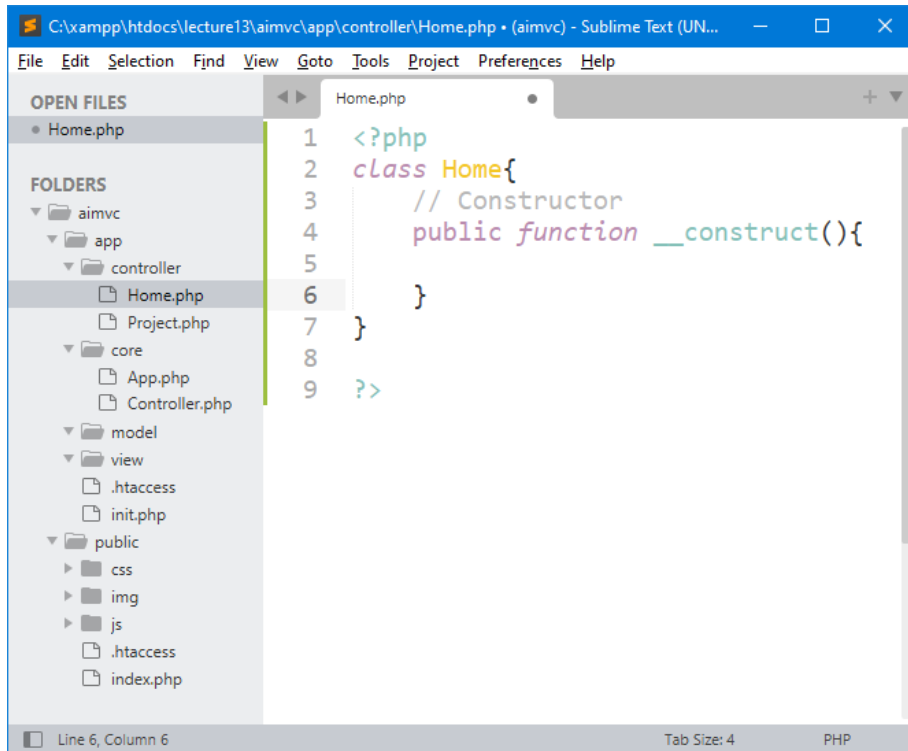
Start *from* Here !!!

A decorative border of green leaves and branches at the top of the slide.

3 – Create Classes (*Home & Project*) as Controller and Default Method

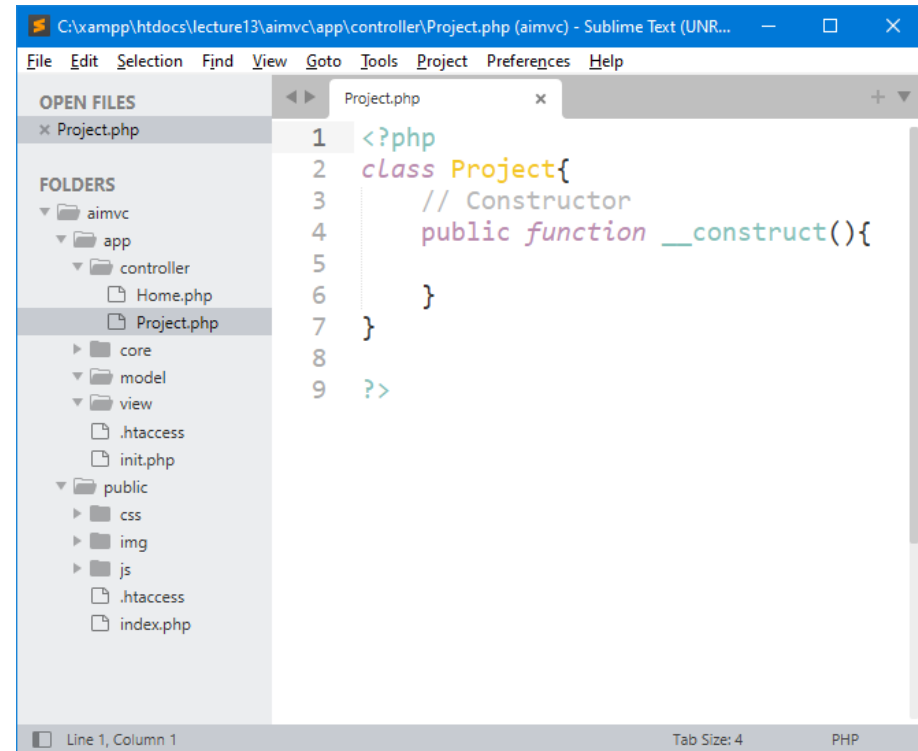
A decorative border of green leaves and branches at the bottom of the slide.

Create Two Controller Classes



```
1 <?php
2 class Home{
3     // Constructor
4     public function __construct(){
5
6     }
7 }
8
9 ?>
```

The screenshot shows the Sublime Text editor with the file 'Home.php' open. The left sidebar displays the project structure, including folders like 'aimvc', 'app', 'controller', 'core', 'model', 'view', and 'public'. The 'controller' folder is expanded, showing 'Home.php' and 'Project.php'. The code in 'Home.php' defines a class 'Home' with a constructor method.



```
1 <?php
2 class Project{
3     // Constructor
4     public function __construct(){
5
6     }
7 }
8
9 ?>
```

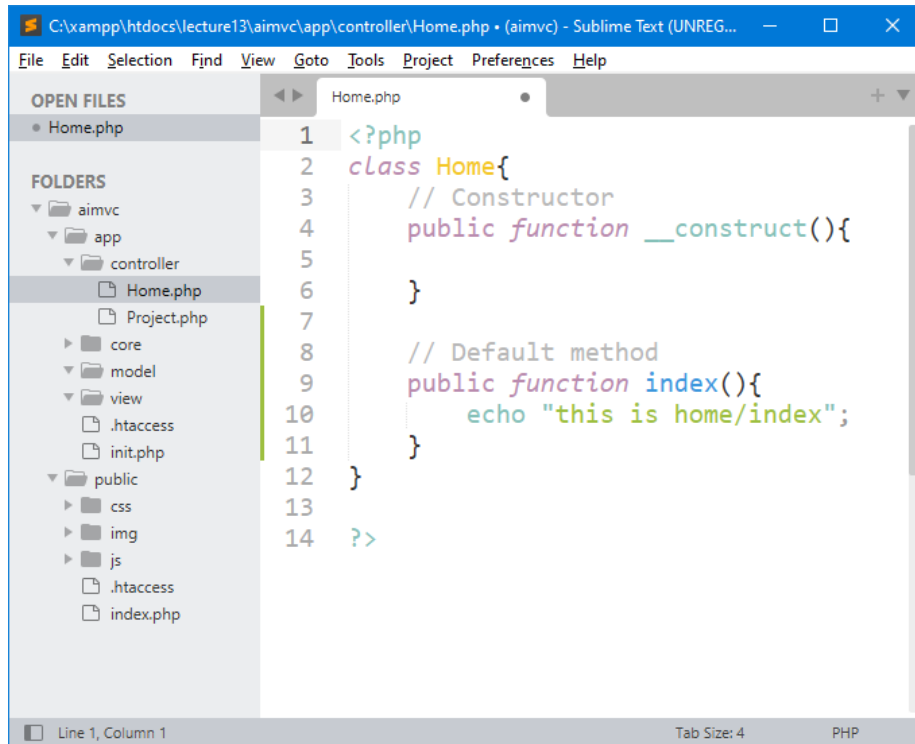
The screenshot shows the Sublime Text editor with the file 'Project.php' open. The left sidebar displays the project structure, including folders like 'aimvc', 'app', 'controller', 'core', 'model', 'view', and 'public'. The 'controller' folder is expanded, showing 'Home.php' and 'Project.php'. The code in 'Project.php' defines a class 'Project' with a constructor method.

Important Note:

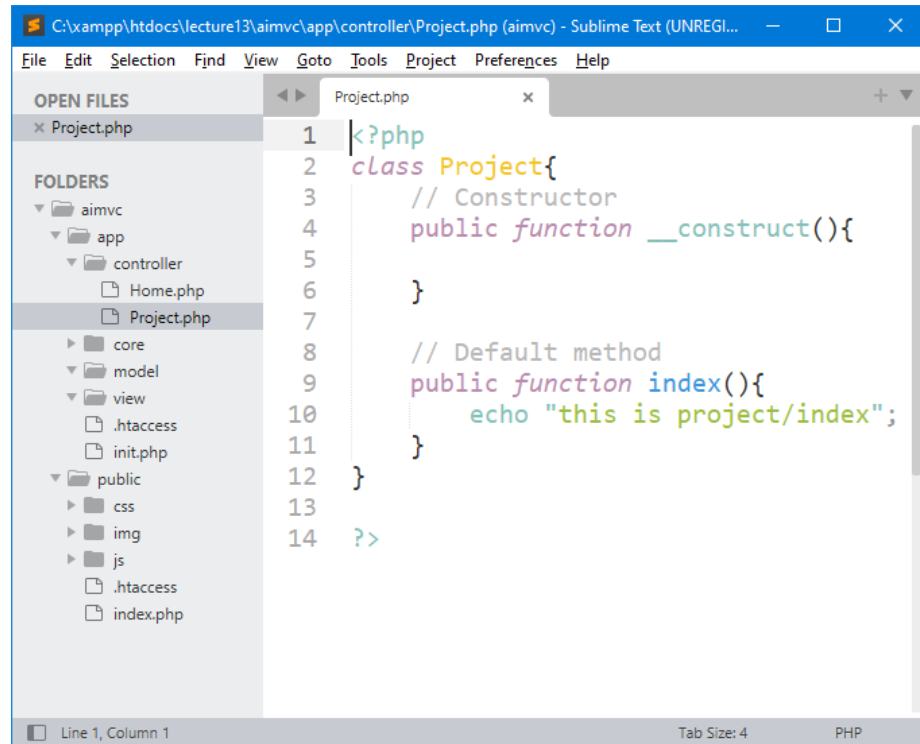
Class names must start with an uppercase letter.

Create *two classes* in *controller* directory to represent/handle **home** and **project** pages in our framework.

index() as a Default Method



```
1 <?php
2 class Home{
3     // Constructor
4     public function __construct(){
5
6     }
7
8     // Default method
9     public function index(){
10         echo "this is home/index";
11     }
12 }
13
14 ?>
```



```
1 <?php
2 class Project{
3     // Constructor
4     public function __construct(){
5
6     }
7
8     // Default method
9     public function index(){
10         echo "this is project/index";
11     }
12 }
13
14 ?>
```

When we do not write/request anything in the URL (“/aimvc/public/”), then this method (*index*) will be called.

__construct() vs index() methods

Why don't you just use the constructor instead of the index method?

__construct() is called first, then according to URL is called index() or other functions.

Public function __construct() should contain:

- 1) allocating resources used in entire class ex. `$this->load`
- 2) check user authentication (*if entire class requires it*)

Public function index() should contain:

- 1) allocating resources used only in this function
- 2) calling views or displaying anything

Link:

<https://stackoverflow.com/questions/16691036/codeigniter-2-difference-between-index-and-construct-and-what-to-put-in-cons>



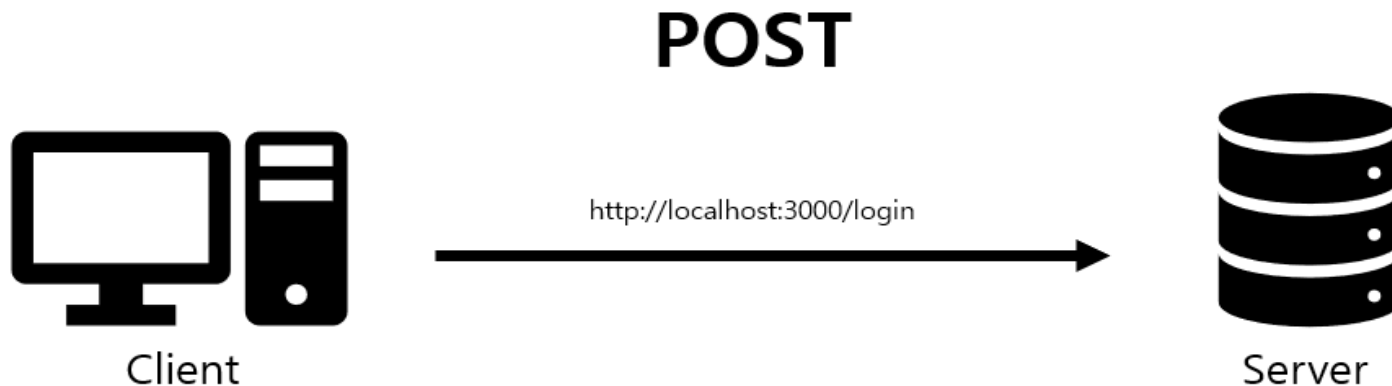
A decorative border of green leaves and branches at the top of the slide.

4 – Handle URL Request *with* Controller Classes (*home & project*)

A decorative border of green leaves and branches at the bottom of the slide.

URL Request POST dan GET adalah dua metode yang berbeda dalam pengiriman data antara client (*biasanya browser*) dan server dalam konteks HTTP (*Hypertext Transfer Protocol*).

- 1) **Method GET:** Digunakan untuk mengambil data dari server. Permintaan GET akan mengirim data dalam URL, yang dapat terlihat oleh semua orang karena informasi dikirimkan dalam string query di URL.
- 2) **Method POST:** Digunakan untuk mengirim data ke server. Permintaan POST akan mengirim data dalam badan permintaan HTTP, yang tidak terlihat dalam URL dan cenderung lebih aman untuk mengirim data sensitif.



Create Global Variables

In PHP class, global variable as property in class to store a **default controller (home)**.

The screenshot shows a Sublime Text editor window titled 'C:\xampp\htdocs\lecture13\aimvc\app\core\App.php (aimvc) - Sublime Text (UNREGISTERED)'. The editor displays the following PHP code:

```
1 <?php
2 // Create a class
3 class App{
4     // Global variables
5     public $controller = "";
6     public $method = "";
7     public $parameter = "";
8
9     // PHP Constructor
10    public function construct(){
11        // Default controller
12        $this->initDefaultController("Home", "index", "");
13
14        // Display structured information (type and value)
15        $url = $this->parseURL();
16        var_dump($url);
17    }
18
19    // Initialize default global variable
20    private function initDefaultController($controller, $method, $param){
21        $this->controller = $controller;
22        $this->method = $method;
23        $this->parameter = $param;
24    }
25
26
27    // Method to Parsing URL Request
28    public function parseURL(){
29        if(isset($_GET['url'])){
30            // Remove end slash
31            $url = rtrim($_GET['url'], '/');
```

Three red boxes highlight specific parts of the code:

- Box 1 (lines 4-7): Global variable declarations: `public $controller = "";`, `public $method = "";`, and `public $parameter = "";`.
- Box 2 (lines 11-12): Constructor call: `$this->initDefaultController("Home", "index", "");`.
- Box 3 (lines 20-24): Initialization function: `private function initDefaultController($controller, $method, $param){`, `$this->controller = $controller;`, `$this->method = $method;`, `$this->parameter = $param;`, and `}`.

An arrow points from the text "Global variables" to the first box. A curved arrow points from the first box to the third box, indicating the flow of data from the global variable declarations to their initialization.

3 selection regions

Tab Size: 4 PHP

Remember this \$url variable !!!

File Edit Selection Find View Goto Tools Project Preferences Help

OPEN FILES

- App.php

FOLDERS

- aimvc
 - app
 - controller
 - Home.php
 - Project.php
 - core
 - App.php
 - Controller.php
 - model
 - view
 - .htaccess
 - init.php
 - public
 - css
 - img
 - js
 - .htaccess
 - index.php

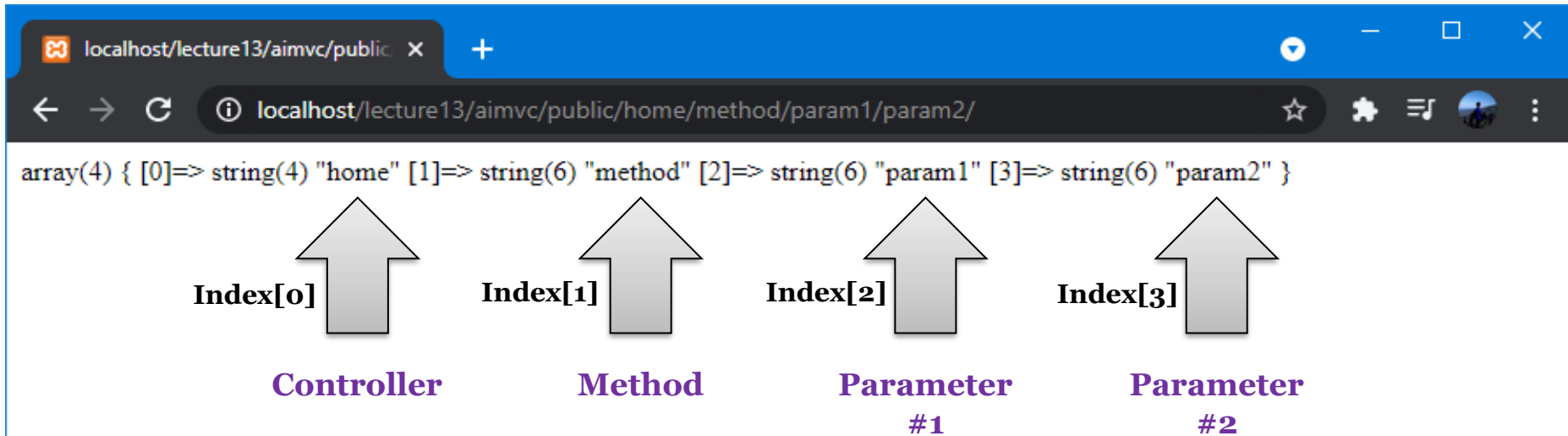
```
1 <?php
2 // Create a class
3 class App{
4     // Global variables
5     public $controller = "";
6     public $method = "";
7     public $parameter = "";
8
9     // PHP Constructor
10    public function __construct(){
11        // Default controller
12        $this->initDefaultController("Home", "index", "");
13
14        // Display structured information (type and value)
15        $url = $this->parseURL();
16        //var_dump($url);
17
18    }
19
20    // Initialize default global variable
21    private function initDefaultController($controller, $method, $
    param){
```

If the **url request** change, we can handle next request after this line of code.

Line 18, Column 10

Tab Size: 4 PHP

After Add Some URI Routing Rules



Our app starts from this directory location:
<http://localhost/lecture13/aimvc/public/>

This means Controller default (*home*) and method default (*index*).

URL Format: <http://localhost/lecture13/aimvc/public/home/method/param1/param2/>

Get element *index 0* as Controller Name

C:\xampp\htdocs\lecture13\aimvc\app\core\App.php • (aimvc) - Sublime Text (UNREGISTERED)

File Edit Selection Find View Goto Tools Project Preferences Help

OPEN FILES

- App.php

FOLDERS

- aimvc
 - app
 - controller
 - Home.php
 - Project.php
 - core
 - App.php
 - Controller.php
 - model
 - view
 - .htaccess
 - init.php
 - public
 - css
 - img
 - js
 - .htaccess
 - index.php

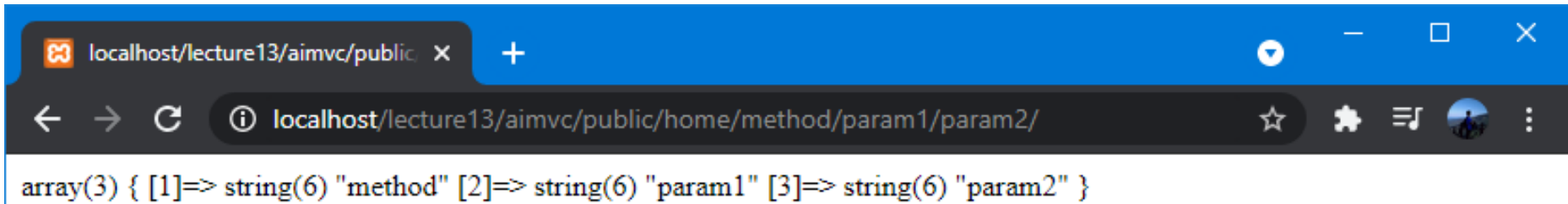
```
1 <?php
2 // Create a class
3 class App{
4     // Global variables
5     public $controller = "";
6     public $method = "";
7     public $parameter = "";
8
9     // PHP Constructor
10    public function __construct(){
11        // Assign to default controller
12        $this->initDefaultController("Home", "index", "");
13
14        // Display structured information (type and value)
15        $url = $this->parseURL();
16        // var_dump($url);
17
18        // Handle controller
19        if(file_exists('../app/controller/'.$url[0].'.php')){
20            // Change controller name
21            $this->controller = $url[0];
22            // Delete element index 0
23            unset($url[0]);
24
25            // Display array
26            var_dump($url);
27        }
28    }
29 }
```

Check if specific controller class file exist in controller directory or not.

20 lines, 494 characters selected

Tab Size: 4 PHP

Recognize Controller Name



There's no **index 0** after `var_dump()`?



Element of *index 0* (value: *home*) already recognized as a controller class name.

Full Handle Controller

Annotation 1 (Left): `if(!is_null($url)){
// code
}`

Annotation 2 (Top Right): After get controller name in controller directory, you can create an object based on controller class that you already included.

Annotation 3 (Bottom Right):

```
// Handle controller  
if(!empty($url)){  
    if(file_exists('../app/controller/'.$url[0].'.php')){  
        // Change controller name  
        $this->controller = $url[0];  
        // Delete element index 0  
        unset($url[0]);  
    }  
}
```

```
require_once '../app/controller/'.$this->controller.'.php';  
$this->controller = new $this->controller; // Instansiasi Object
```

```
public function __construct(){  
    // Assign to default controller  
    $this->initDefaultController("Home", "index");  
  
    // Display structured information (type and value)  
    $url = $this->parseURL();  
    //var_dump($url);  
  
    // Handle controller  
    if(file_exists('../app/controller/'.$url[0].'.php')){  
        // Change controller name  
        $this->controller = $url[0];  
        // Delete element index 0  
        unset($url[0]);  
  
        // Display array  
        var_dump($url);  
    }  
  
    require_once '../app/controller/'.$this->controller.'.php';  
    $this->controller = new $this->controller; // Instansiasi Object  
  
    // Initialize default global variable  
    private function initDefaultController($controller, $method, $param){  
        $this->controller = $controller;  
        $this->method = $method;  
        $this->parameter = $param;  
    }  
}
```

13 lines, 372 characters selected

Tab Size: 4

PHP

A decorative border of green leaves and branches at the top of the page.

5 – Handle URL Request with ***Method*** and ***Default Method***

A decorative border of green leaves and branches at the bottom of the page.

Recognize Method Name (default or not)

The screenshot shows a Sublime Text editor window with the file `C:\xampp\htdocs\lecture13\aimvc\app\core\App.php` open. The editor displays PHP code for a controller. A red box highlights a section of code starting at line 32, which handles a method request. A callout box with an arrow pointing to the `if(isset($url[1]))` condition explains its logic.

Array index in url[1] contains method name from input request url. After checking the existence of method, this system can decide to output default method or not.

```
18 // Handle controller
19 if(file_exists('../app/controller/'.$this->controller.'.php')){
20     // Change controller name
21     $this->controller = $url[0];
22     // Delete element index 0
23     unset($url[0]);
24
25     // Display array
26     var_dump($url);
27 }
28
29 require_once '../app/controller/'.$this->controller.'.php';
30 $this->controller = new $this->controller; // Instansiasi Object
31
32 // Handle Method
33 if(isset($url[1])){
34     $name_method = $url[1]; // move method name
35     if(method_exists($this->controller, $name_method)){
36         // Change method name
37         $this->method = $name_method;
38         // Delete element index 1 in array
39         unset($url[1]);
40     }
41 }
42
43 // Initialize default global variable
44 private function initDefaultController($controller, $method, $param){
45
```

10 lines, 279 characters selected

Tab Size: 4

PHP

Test Handle Method

The screenshot shows a web browser window with the address bar displaying `localhost/lecture13/aimvc/public/home/index/param1/param2/`. Below the address bar, the text `array(2) { [2]=> string(6) "param1" [3]=> string(6) "param2" }` is shown. Two arrows point from the text below to the `[2]` and `[3]` indices in the array. The first arrow is labeled **Index[2]** and **Parameter #1**. The second arrow is labeled **Index[3]** and **Parameter #2**. A callout box in the bottom right corner contains the text: **Element of *index 1* (value: *index*) already recognized as a *default method name* (*index*).**

localhost/lecture13/aimvc/public x +

localhost/lecture13/aimvc/public/home/index/param1/param2/

array(2) { [2]=> string(6) "param1" [3]=> string(6) "param2" }

Index[2] Index[3]

Parameter #1 Parameter #2

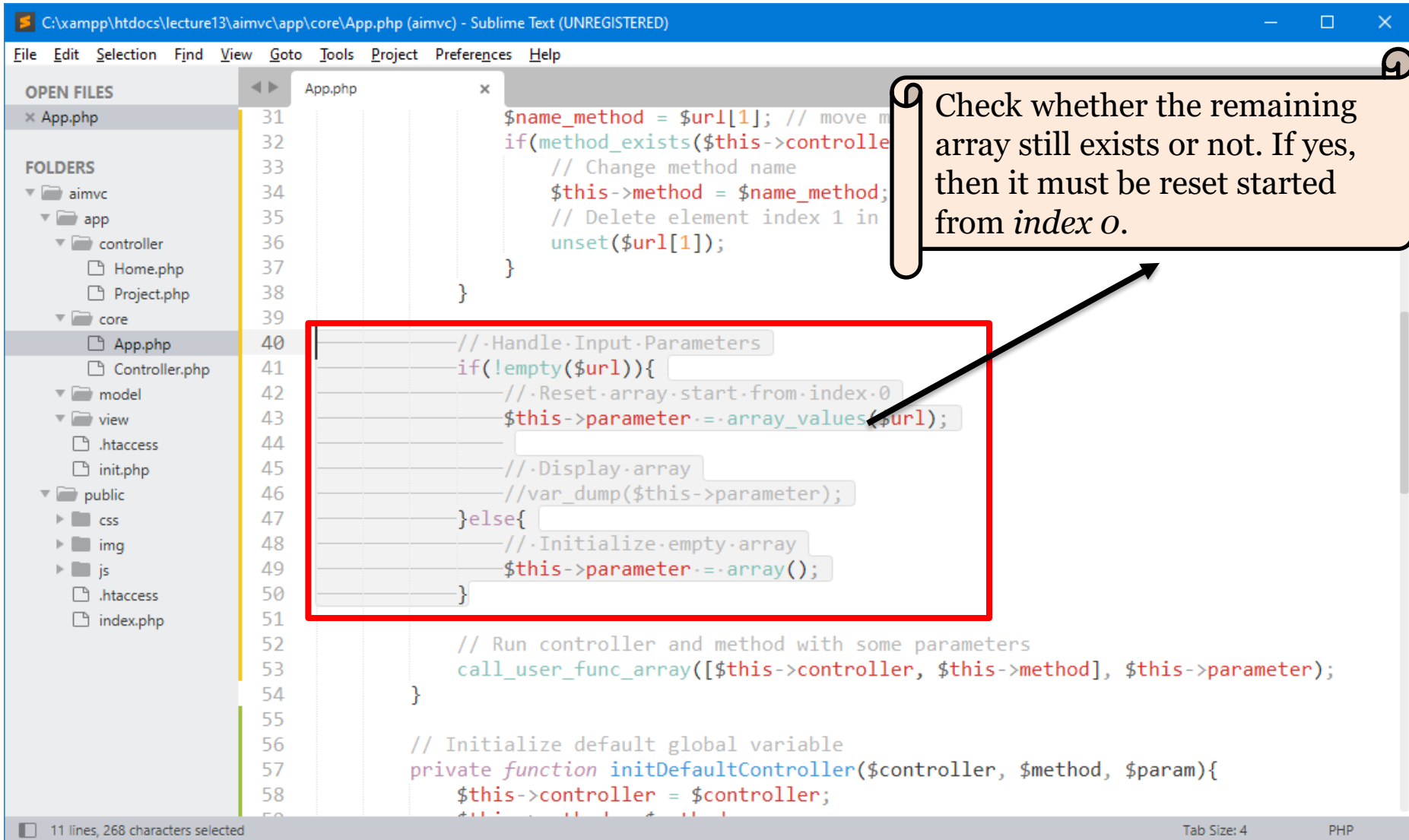
Element of *index 1* (value: *index*) already recognized as a *default method name* (*index*).

A decorative border of green leaves and branches at the top of the slide.

6 – Handle URL Request with *Parameter*

A decorative border of green leaves and branches at the bottom of the slide.

Recognize Input Parameters



C:\xampp\htdocs\lecture13\aimvc\app\core\App.php (aimvc) - Sublime Text (UNREGISTERED)

File Edit Selection Find View Goto Tools Project Preferences Help

OPEN FILES

- App.php

FOLDERS

- aimvc
 - app
 - controller
 - Home.php
 - Project.php
 - core
 - App.php
 - Controller.php
 - model
 - view
 - .htaccess
 - init.php
 - public
 - css
 - img
 - js
 - .htaccess
 - index.php

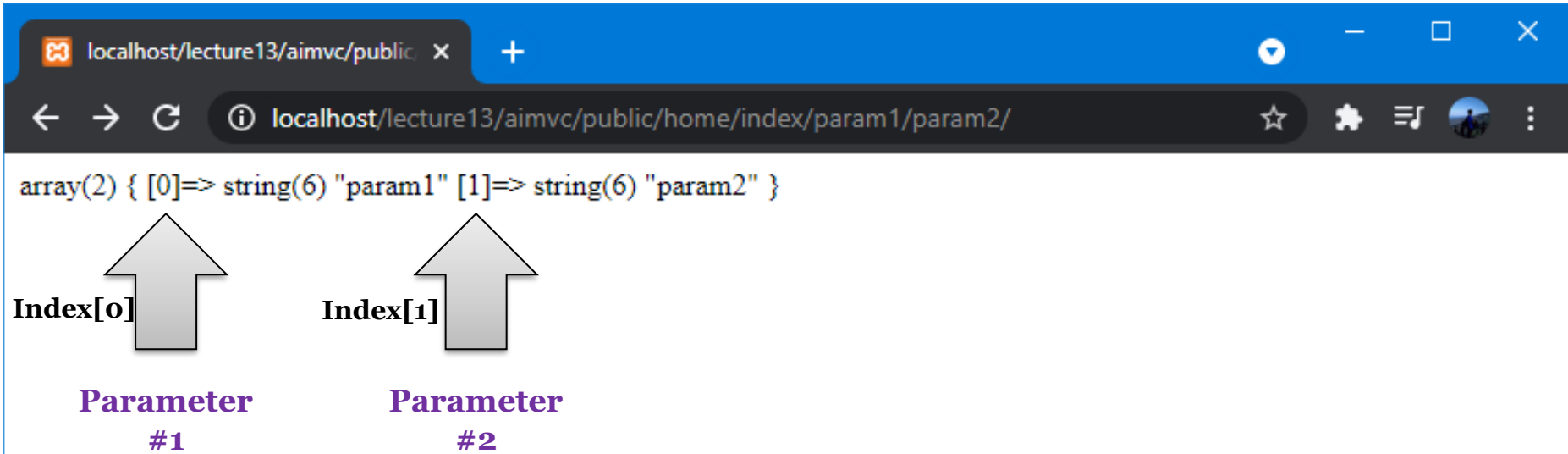
```
31 $name_method = $url[1]; // move m
32 if(method_exists($this->controller
33 // Change method name
34 $this->method = $name_method;
35 // Delete element index 1 in
36 unset($url[1]);
37 }
38 }
39
40 // Handle Input Parameters
41 if(!empty($url)){
42 // Reset array start from index 0
43 $this->parameter = array_values($url);
44
45 // Display array
46 // var_dump($this->parameter);
47 }else{
48 // Initialize empty array
49 $this->parameter = array();
50 }
51
52 // Run controller and method with some parameters
53 call_user_func_array([$this->controller, $this->method], $this->parameter);
54 }
55
56 // Initialize default global variable
57 private function initDefaultController($controller, $method, $param){
58 $this->controller = $controller;
```

Check whether the remaining array still exists or not. If yes, then it must be reset started from *index 0*.

11 lines, 268 characters selected

Tab Size: 4 PHP

Test Handle Input Parameters



The screenshot shows a web browser window with the address bar displaying `localhost/lecture13/aimvc/public/home/index/param1/param2/`. The page content shows a PHP array: `array(2) { [0]=> string(6) "param1" [1]=> string(6) "param2" }`. Below the array, two arrows point from the labels `Index[0]` and `Index[1]` to the first and second elements of the array, respectively. These labels are identified as **Parameter #1** and **Parameter #2**.

array(2) { [0]=> string(6) "param1" [1]=> string(6) "param2" }

Index[0] Index[1]

Parameter #1 **Parameter #2**

All elements in the array containing the "*parameters*" have been reset and are now stored in global variables.

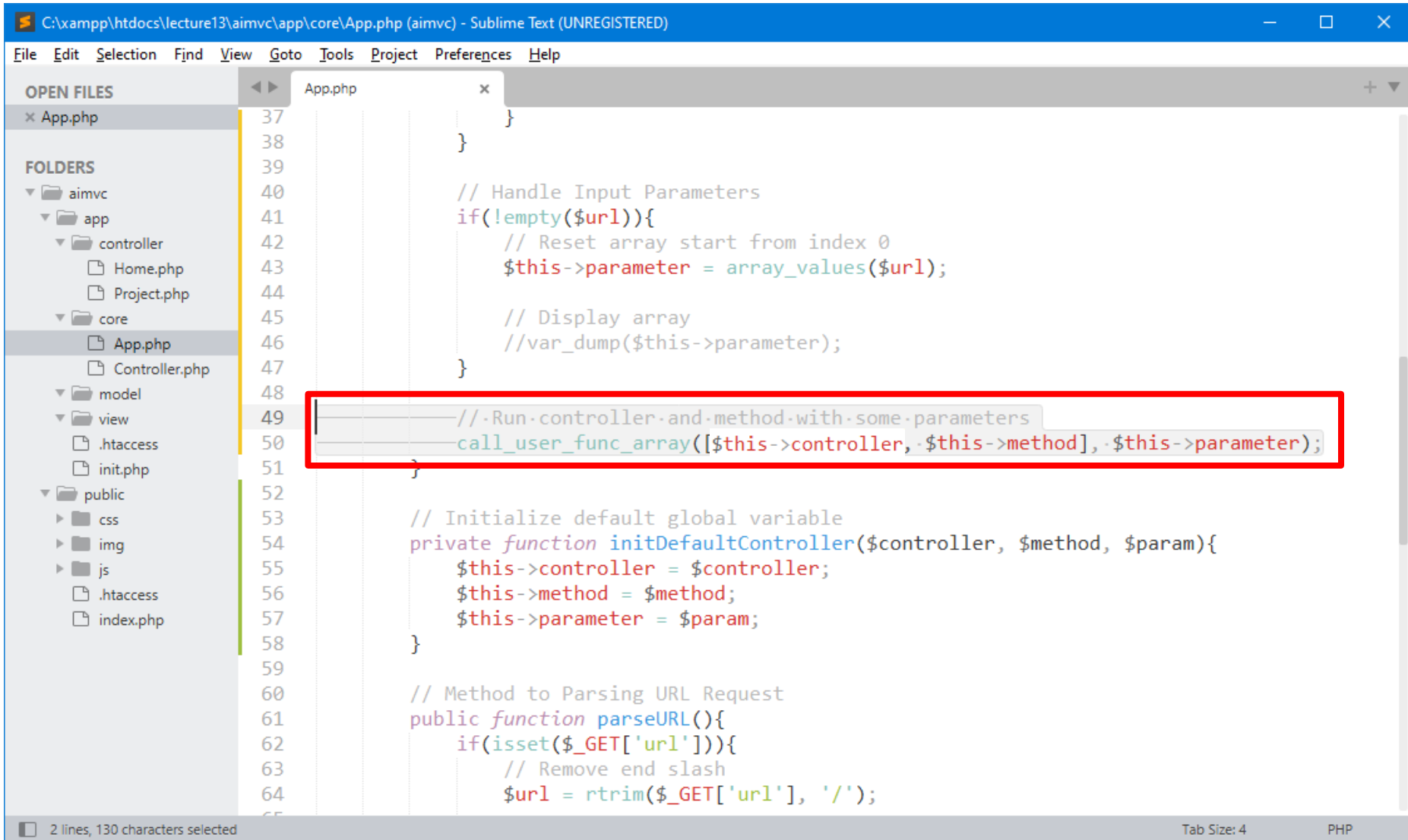
A decorative border of green leaves and branches at the top of the slide.

7 – Finalize URL Request

[run controller & method with parameters]

A decorative border of green leaves and branches at the bottom of the slide.

Run controller & method



C:\xampp\htdocs\lecture13\aimvc\app\core\App.php (aimvc) - Sublime Text (UNREGISTERED)

File Edit Selection Find View Goto Tools Project Preferences Help

OPEN FILES

- App.php

FOLDERS

- aimvc
 - app
 - controller
 - Home.php
 - Project.php
 - core
 - App.php
 - Controller.php
 - model
 - view
 - .htaccess
 - init.php
 - public
 - css
 - img
 - js
 - .htaccess
 - index.php

```
37 }
38 }
39
40 // Handle Input Parameters
41 if(!empty($url)){
42     // Reset array start from index 0
43     $this->parameter = array_values($url);
44
45     // Display array
46     //var_dump($this->parameter);
47 }
48
49 //Run controller and method with some parameters
50 call_user_func_array([$this->controller, $this->method], $this->parameter);
51 }
52
53 // Initialize default global variable
54 private function initDefaultController($controller, $method, $param){
55     $this->controller = $controller;
56     $this->method = $method;
57     $this->parameter = $param;
58 }
59
60 // Method to Parsing URL Request
61 public function parseURL(){
62     if(isset($_GET['url'])){
63         // Remove end slash
64         $url = rtrim($_GET['url'], '/');
```

2 lines, 130 characters selected

Tab Size: 4

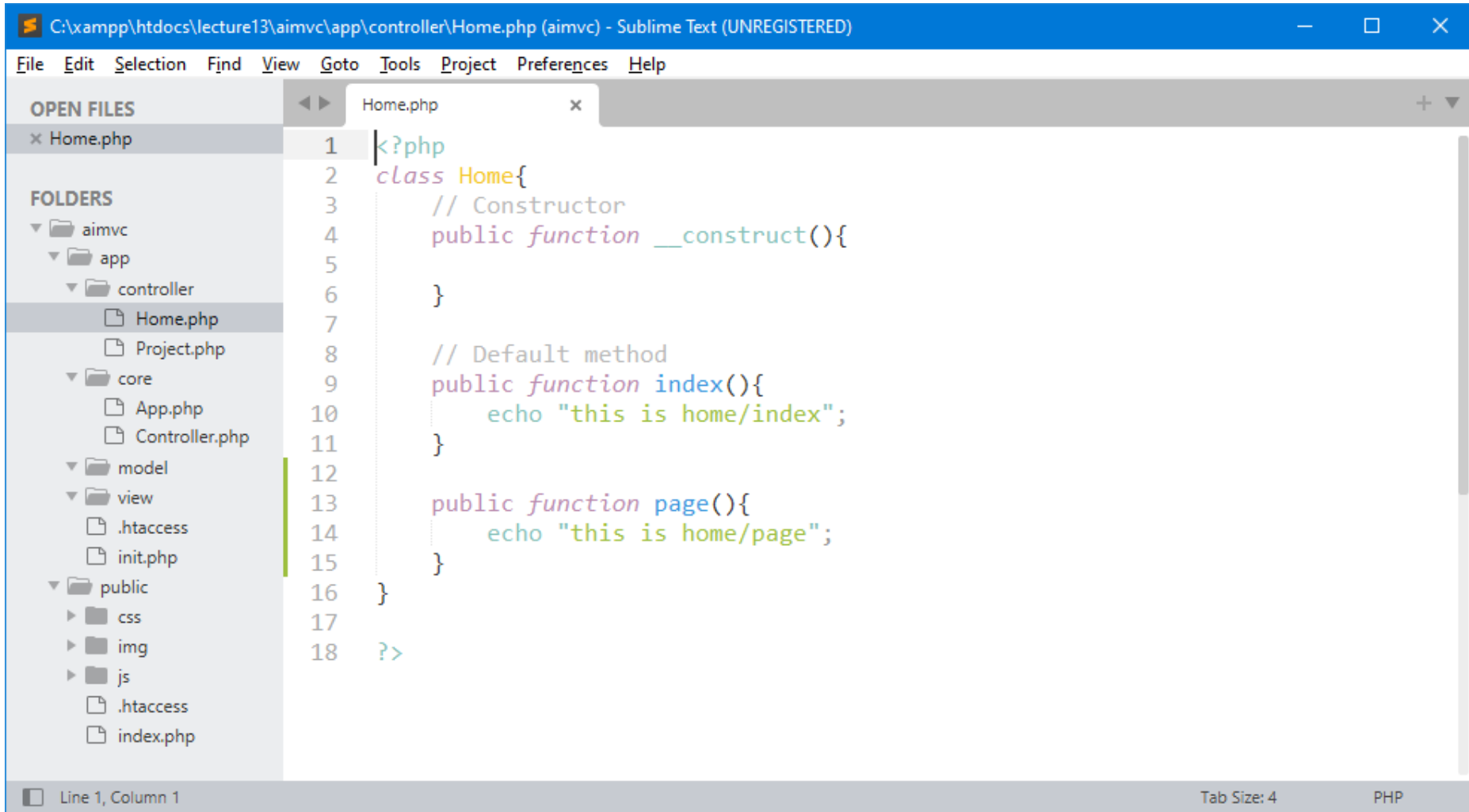
PHP



8 – Testing

Back-end Framework

Add More Methods



```
C:\xampp\htdocs\lecture13\aimvc\app\controller\Home.php (aimvc) - Sublime Text (UNREGISTERED)
```

File Edit Selection Find View Goto Tools Project Preferences Help

OPEN FILES

- × Home.php

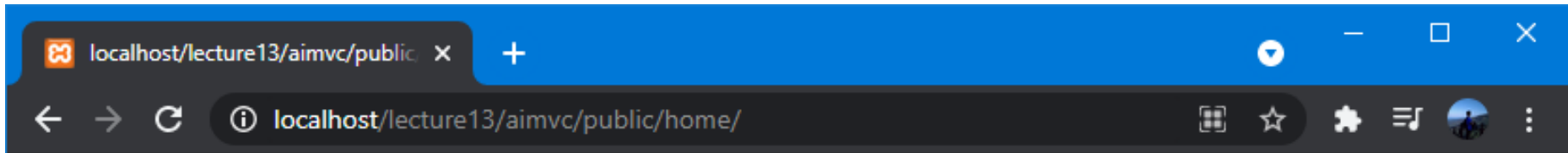
FOLDERS

- aimvc
 - app
 - controller
 - Home.php
 - Project.php
 - core
 - App.php
 - Controller.php
 - model
 - view
 - .htaccess
 - init.php
 - public
 - css
 - img
 - js
 - .htaccess
 - index.php

```
1 <?php
2 class Home{
3     // Constructor
4     public function __construct(){
5
6     }
7
8     // Default method
9     public function index(){
10         echo "this is home/index";
11     }
12
13     public function page(){
14         echo "this is home/page";
15     }
16 }
17
18 ?>
```

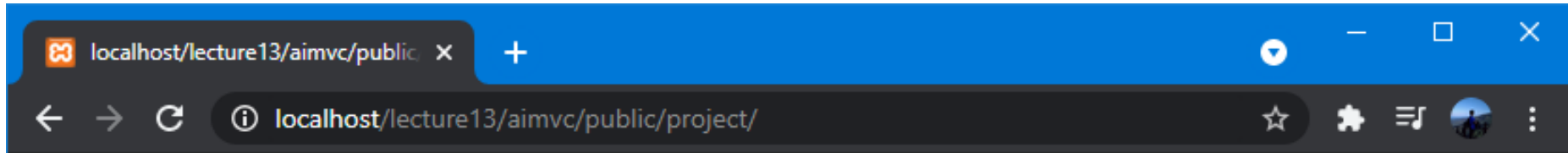
Line 1, Column 1 Tab Size: 4 PHP

Access Home Controller (*index*)



**Controller home yang di request.
Jika halaman yang dicari/request
tidak ditemukan maka method
yang dipanggil adalah method
default yaitu index().**

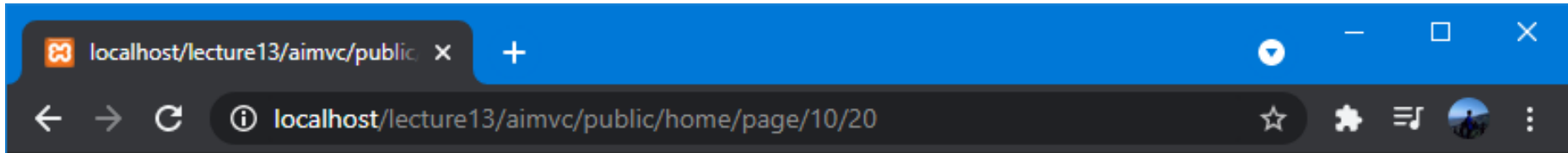
Access Project Controller (*index*)



this is project/index

**Controller Project yang di request.
Jika halaman yang dicari/request
tidak ditemukan maka method yang
dipanggil adalah method default
yaitu index().**

Access Home Controller *with* Specific Method



this is home/page

Controller Home dan method page yang di request.



Exercise *for* Students

Latihan untuk *Students*

Create directory : C:\xampp\htdocs\lecture13\

Create file : `_.php`

- ☐ Ikuti semua step dan buat implementasikan dari codes yang dibahas sebelumnya.
- ☐ Follow all the steps and make an implementation of the previously discussed codes.

1) Sudah terdapat 2 controller yang dibuat yaitu **Home** dan **Project**. Tambahkan controller baru dengan nama **Help** dan **About** controller.

2) Tambah page (method di dalam class controller) yang baru untuk masing-masing controller berikut ini:

Home Controller:

- index()
- page()
- profile()

Project Controller:

- index()
- personal()
- group()

Help Controller:

- index()
- guide()

About Controller:

- index()
- company()
- mission()

Reference (#1)

Controllers

<https://codeigniter.com/userguide3/general/controllers.html#id1>

URI Routing

<https://codeigniter.com/userguide3/general/routing.html>

Model View Controller example

<https://developer.mozilla.org/en-US/docs/Glossary/MVC>

Learn About MVC Architecture

<https://www.c-sharpcorner.com/article/learn-about-mvc-architecture/>

MVC URL Routing

<https://sudhersonv.wixsite.com/codeblogs/single-post/2015/05/19/mvc-url-routing>

Model View Controller Pattern – MVC Architecture and Frameworks

<https://www.freecodecamp.org/news/the-model-view-controller-pattern-mvc-architecture-and-frameworks-explained/>

Apa itu MVC? Mengenal Mvc (Model View Controller)

<https://note-ade.blogspot.com/2020/10/mvc.html>

Sistem Pemrograman Model View Controller (MVC)

<https://informatika.uc.ac.id/id/2016/12/sistem-pemrograman-model-view-controller-mvc/>

A Beginner's guide to Ruby on Rails MVC

<https://hackernoon.com/beginners-guide-to-ruby-on-rails-mvc-model-view-controller-pattern-4z19196a>

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

